AADK TASK 3
Compose Layout:Text And Images
Name: Lakshmitha R

Date: [25/02/2026]

Task 1– Text Composables Understanding & Exploration

1. Understanding @Composable Functions

A `@Composable` function in Jetpack Compose is a special Kotlin function used to define and display UI components. Instead of using XML layouts, UI is written directly in Kotlin using composable functions. These functions describe how the UI should look and automatically update when data changes.

2. Key Text and Layout Composables

a) Text

- **What it does:**
  Displays text on the screen such as titles, labels, or paragraphs.
- **Example use-case:**
  Showing a greeting message like “Hello Lakshmitha” in a greeting card app.

b) Column

- **What it does:**
  Arranges UI elements vertically from top to bottom.
- **Example use-case:**
  Login screen layout where Title, Email field, Password field, and Button are stacked vertically.

c) Row

- **What it does:**
  Arranges UI elements horizontally from left to right.
- **Example use-case:**
  Placing two buttons side-by-side such as “Accept” and “Cancel”.

3. Role of Column and Row in Text Layout

Column and Row control how Text elements are positioned on the screen.

- **Column** displays Text components one below the other, creating a vertical flow.
- **Row** places Text elements next to each other horizontally.

By choosing the correct layout composable, the UI becomes organized, readable, and responsive across different screen sizes.

4. Wireframe Design Description

The designed screen layout includes the following elements:

- **Title:** Large text at the top using `Text` inside a `Column`
- **Subtitle:** Smaller text below the title using another `Text`
- **Paragraph:** A longer description text placed below the subtitle
- **Two Buttons:** Placed side-by-side using a `Row` composable

Composable Structure Used:

- `Column` → Overall vertical layout
- `Text` → Title, subtitle, and paragraph
- `Row` → Horizontal alignment of two text buttons

This wireframe helps visualize the UI before implementation.

5. Reflection

Planning the text layout visually before coding is very useful because it gives a clear idea of how UI elements should be arranged. It reduces confusion during development and avoids repeated changes in code. Visual planning also helps in selecting the right composables such as Column or Row. Overall, it saves time and leads to a cleaner and more structured UI design.

Task 2 – Image Inclusion & Resource Management in Compose

1. Image Composable in Jetpack Compose

The `Image` composable in Jetpack Compose is used to display images in an Android application. It allows developers to show images from local drawable resources or remote sources. The Image composable works together with layout composables like Column, Row, and Box to position images properly on the screen.

Example Use-Case:
Displaying a profile picture at the top of a profile screen or showing an image banner in a greeting card app.

2. Importance of Image Density in Android

Image density is important in Android because devices have different screen resolutions and pixel densities. Using correct density resources such as drawable-nodpi helps maintain image quality and performance across devices.

If image density is ignored:

- Images may appear blurry or stretched
- App size may increase unnecessarily
- Performance may be affected due to large unoptimized images

Using optimized images ensures smooth UI rendering and better user experience.

3. Best Practices for Adding Images in Compose

1. **Proper Naming Convention:**
   Use lowercase letters and underscores (e.g., `profile_image.png`) to avoid resource errors.
2. **Image Size Optimization:**
   Compress images and avoid using very large image files to improve performance and reduce app size.

4. Visual Storyboard Description

- **Frame 1:** Screen with placeholder text only using `Text` inside a `Column`
- **Frame 2:** Same screen with an image added at the top using `Image` composable
- **Frame 3:** Image adjusted to full width using modifiers like `fillMaxWidth()` and cropped for better layout

Each frame demonstrates how adding and adjusting images affects UI structure and appearance.

5. Reflection

Images add visual appeal but also introduce challenges such as performance issues, layout imbalance, and accessibility concerns. Large images can slow down the app, and missing content descriptions affect accessibility. Proper planning helps avoid these issues and ensures a smooth user experience.

Task 3 – Planning a Simple Layout with Text + Image

1. App Concept

Concept Chosen: Birthday Greeting Card
Purpose:
To display a greeting message with an image and text in a visually appealing layout.

2. Screen Layout Description

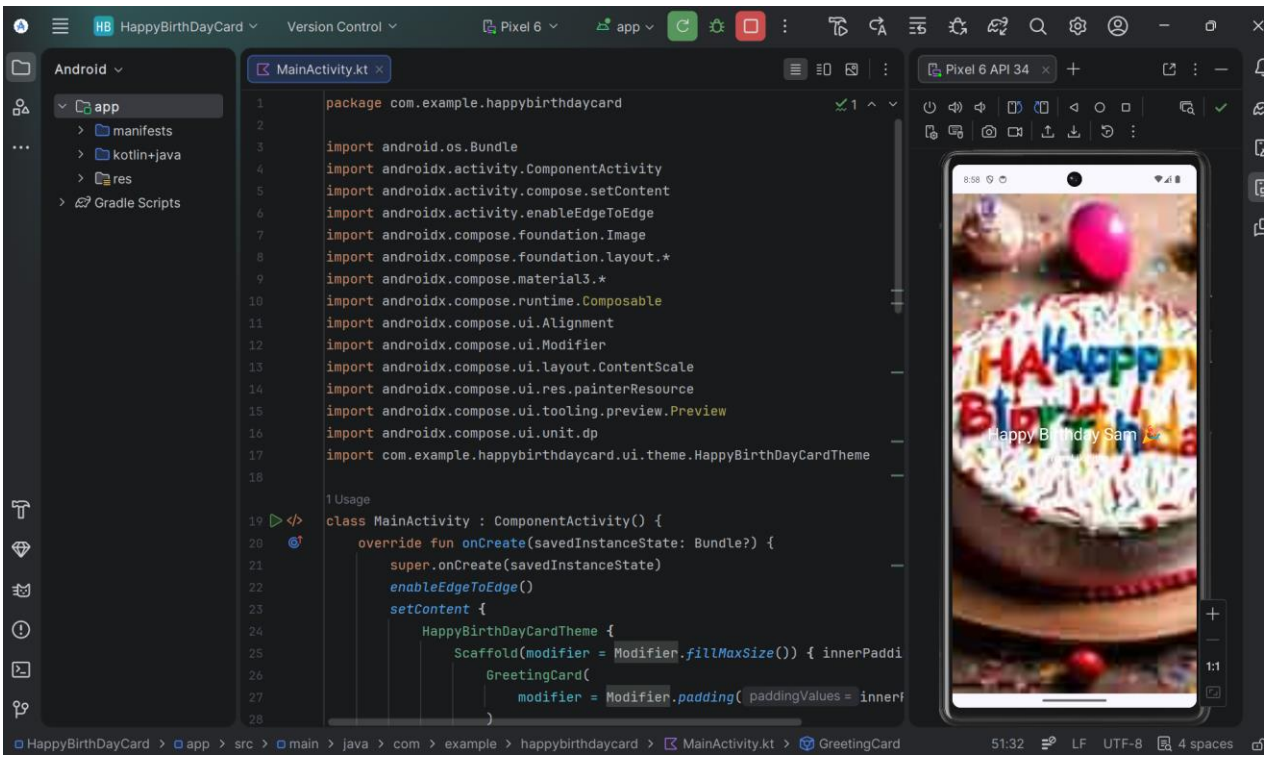
The screen design includes:

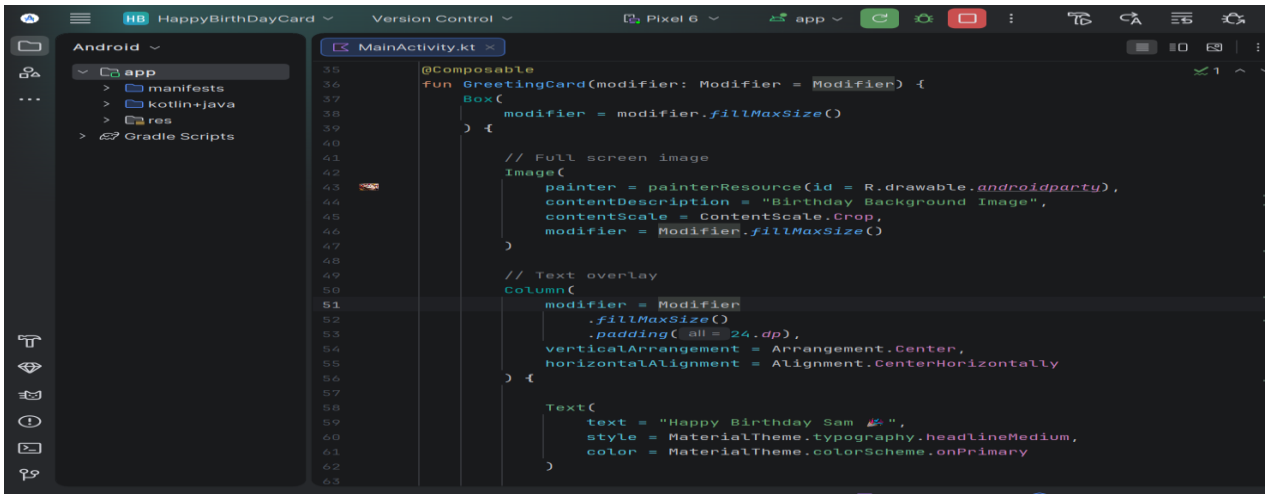
- An image at the top (birthday illustration)
- A title text (“Happy Birthday”)

- A subtitle text (person's name)
- A short greeting message
- A button at the bottom

Composable Structure:

- `Column` → Overall vertical layout
- `Image` → Birthday image
- `Text` → Title, subtitle, and message
- `Row` → Button alignment





3. Spacing and Padding Plan

- 16 dp space between image and title
- 8 dp between title and subtitle
- 8 dp between text blocks
- 24 dp padding around the screen edges

4. Resource Plan

Image Resources

Image	File Name	Folder	Size
Birthday Image	birthday_card.png	drawable-nodpi	Medium

Text Elements

- Title: 24 sp, bold, primary color
- Subtitle: 18 sp, medium weight
- Body text: 16 sp, normal weight

5. Reflection

Planning composables and resources before coding saves development time and reduces layout errors. It ensures correct spacing, better alignment, and consistent design. Pre-planning also helps select optimized image resources, improving performance and UI quality.

Task 4 – Accessibility, Performance & Layout Review

1. Accessibility & Performance Evaluation

The planned Birthday Greeting Card layout was reviewed for usability, accessibility, and performance.

Review Table

Element	Potential Issue	Proposed Fix	Priority
Imageat top	Large image file size	Compress image and resize to 1080×720	High
Title Text	Might be small on some devices	Increase font size to 24 sp	Medium
Text Contrast	Light background reduces readability	Use darker text color	High
Button	Small touch area	Set minimum size to 48 dp	High

2. Mini Action Plan

Before Coding:

- Optimize image size
- Decide font sizes and colors
- Plan spacing and layout structure

During Coding:

- Add `contentDescription` for images
- Use responsive modifiers like `fillMaxWidth()`
- Ensure minimum touch target sizes

After Testing:

- Test on multiple screen sizes
- Adjust spacing and font sizes if required

3. Reflection

Planning accessibility and performance early improves overall app quality. It ensures the UI is readable, responsive, and usable for all users. Early review reduces rework and helps create inclusive and efficient Android applications.

Conclusion

Through this internship work, I gained a clear understanding of how Jetpack Compose is used to design modern Android user interfaces by planning layouts before coding. I learned how to structure text and images using composable functions, manage image resources properly, and combine UI elements into a clean and responsive screen layout. I also understood the importance of spacing, modifiers, accessibility, and performance considerations such as readable text, optimized images, and proper touch target sizes. Overall, this experience improved my confidence in Android app development and helped me develop a design-first mindset for building user-friendly and efficient Compose applications.

Submitted by: Lakshmitha R
MindMatrix Internship - Android App Development using Gen AI
February 2026